Dynamic Firewall with Security Event Logger

SDN + TCP Socket Programming Mini Project (UE25CS243A) - Project Overview

1. Summary

A TCP application runs on an emulated network protected by a firewall that reconfigures itself while the network is running. Clients send messages to a central **Security Event Logging Server**. The server **classifies every connection by source IP, destination IP and port** against a policy file and tells a **Ryu SDN controller** (REST API) to install one of three kinds of OpenFlow rule: **allow**, **deny** or **block**. Rules are removed automatically after a recovery period, and re-installed if the controller restarts, so the network heals without an operator. The controller also polls the switch's flow statistics, so the system can measure how much traffic each rule dropped and optionally detect abusive hosts from switch measurements.

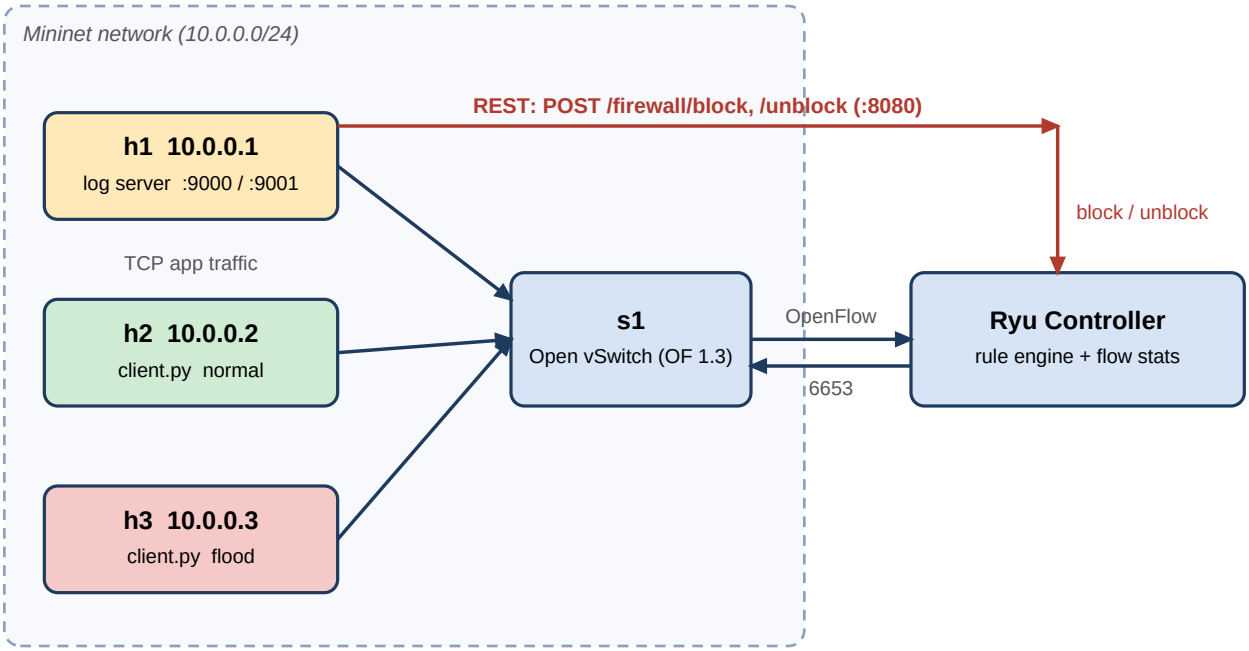
It corresponds to Project #1 in the project list (TCP) and covers the project expectations: TCP client/server, classification by source, destination and port, dynamic firewall rules from the SDN controller, a centralised security-event logger, multiple clients, automatic blocking and recovery, and measurement of response time and network impact against a baseline.

2. The three rule types

Rule	Priority / action	Triggered by	Removed
BLOCK	100, drop everything from one source IP	More than 8 messages in 5 s to a rate-limited service (port 9000); or, optionally, a source the switch measures above a bandwidth threshold	After 15 s of silence
DENY	200, drop one flow (src, dst, TCP/UDP, port)	A source not on the allow-list connects to a protected service (port 9001)	After 30 s
ALLOW	300, skip the drop rules and continue to forwarding	Source listed in trusted_sources (h2): exempt from rate limiting; overrides deny and block	Stays while the server runs

The switch uses a two-table OpenFlow 1.3 pipeline. Table 0 (policy) holds these rules; higher priority wins. Traffic that no rule drops continues to table 1 (forwarding), where the controller's learned MAC flows live.

3. Architecture



h1 hosts the log server. h2 is a trusted, well-behaved client and h3 is the flooding / unauthorised client. s1 talks to the Ryu controller over OpenFlow (port 6653). The log server reaches the controller's REST API (port 8080), which runs in the VM's root namespace, through a NAT node at 10.0.0.254.

4. Components

Component	Role	Technology
Mininet	Virtual network: 3 hosts, 1 switch, NAT node; optional bandwidth shaping for the experiments.	Mininet Python API, TCLink
Open vSwitch (s1)	Forwarding plane. Two-table pipeline; enforces the drop and allow rules.	OVS, OpenFlow 1.3
Ryu controller	Control plane. L2 learning, rule engine (allow / deny / block), flow-statistics poller, REST API.	Ryu, webob
Log server	Threaded TCP server and policy engine: classifies traffic, detects floods and violations, calls the controller, recovers rules, controller circuit breaker, monitor-only mode.	Python socket, threading
Client	Normal and flood traffic: configurable rate, message size, connections, duration; reconnects like a persistent attacker.	Python socket
Policy file	Ports, services, allow-lists, trusted hosts, rate limits.	JSON
Experiments	Static-vs-dynamic harness and analysis (tables and charts).	Python, iperf, ping, matplotlib

5. Files

File	Purpose
controller/firewall_controller.py	Ryu app: pipeline setup, learning switch, rule engine with strict removal, flow-stats polling, REST API
app/log_server.py	TCP server and policy engine; modes: enforce, --no-block (baseline), optional --bw-threshold-mbps
app/client.py	TCP client (normal / flood; --rate, --size, --connections, --duration, --persist)
config/policy.json	Policy: services, allow-lists, trusted sources, rate limits
topology/firewall_topo.py	Mininet topology; build_net() reused by the experiments
experiments/run_experiments.py, analyze.py	Baseline-vs-dynamic experiments and result tables / charts
tests/	Unit tests, parser tests and a localhost end-to-end test
scripts/	install_deps.sh (Mininet, OVS, Ryu for Ubuntu 22.04) and run_tests.sh

6. REST API (controller, port 8080)

Endpoint	Body	Effect
GET /firewall/status	-	Blocked IPs and rule count
POST /firewall/block, /unblock	{"ip": "10.0.0.3"}	Add / remove a block rule
POST /firewall/rule	{"action", "src", "dst", "proto", "dport"}	Add an allow, deny or block rule (duplicates ignored)
POST /firewall/rule/remove	{"id": n} or the add body	Remove exactly one rule (strict match)
GET /firewall/rules	-	All rules with packet and byte counters
GET /firewall/stats	-	Per-source rates, packets dropped, connected switches

7. Event flows

Flood from h3 on port 9000 (BLOCK):

Step	What happens
1	h3 sends messages about every 0.1 s; the log server logs each one and counts them per source IP in a 5 s window.
2	The 9th message exceeds the threshold. The server logs an ALERT and calls POST /firewall/block.
3	The controller adds a priority-100 drop rule to table 0 of s1 and records it.
4	h3's traffic is dropped by the switch; its TCP connection stalls and times out. h2 is unaffected.
5	After 15 s without messages from h3 the server calls POST /firewall/unblock; the controller deletes exactly that rule.

Unauthorised access from h3 to port 9001 (DENY): the server sees the connection, finds h3 is not on the allow-list, replies DENIED and installs a priority-200 rule for that one flow (h3 to h1, TCP, port 9001). h3 can still use port 9000. The rule is removed after 30 s. h2 (trusted) is allowed on 9001.

8. Evaluation method

The experiment harness measures what a **legitimate host (h2)** experiences while h3 floods, on a network where the server link is limited to 10 Mbit/s. Three conditions are compared: **idle** (no flood), **static** (baseline: the flood runs and the log server only monitors, so the network never reacts) and **dynamic** (the SDN block is active). Three flood intensities are used: low (1 connection, 20 msg/s), medium (1 connection, unthrottled) and high (4 connections, unthrottled), each repeated several times.

Measured: h2 throughput (iperf), h2 latency and loss (ping), traffic the attacker got through, time to the first block, and packets and bytes dropped by the switch's rules (flow statistics). The flood client reconnects after being blocked, like a persistent attacker. Results are produced by experiments/analyze.py (summary.md and charts).

Status: the harness and analysis are written and their parsing and table / chart code was tested on sample data, but the experiments have not yet been run on Mininet. Insert the generated results/summary.md and charts here after running them.

9. Testing

- **Controller logic (17 tests):** rule engine, priorities, strict removal, validation, learning switch, two-table pipeline; real OpenFlow messages are built and serialised against a simulated switch; flow-statistics maths.
- **Log server policy (11 tests):** classification, rate threshold, deny, monitor-only, controller-down, bandwidth detection, reconciliation with the controller.
- **End to end on localhost (9 tests):** real Ryu controller, log server and clients: block and recovery, deny, trusted allow rule, monitor-only baseline, controller failure, rules restored after a controller restart, port-in-use failure, load options.
- **Not covered by automated tests:** real Open vSwitch behaviour (drop rules discarding packets, NAT node, bandwidth shaping). These are checked by running the demo scenarios on Mininet.

10. Rubric coverage

Rubric item	Where it is addressed
Problem understanding & architecture	Sections 1-3, README, docs/VIVA_NOTES.md
TCP socket implementation	Threaded log server on two ports, client with several modes, timeouts, application replies
Mininet + SDN implementation	Topology; Ryu two-table pipeline with allow / deny / block rules
Socket-SDN integration	Log server drives the controller via REST; controller flow statistics feed back to the log server
Complete functionality	Classification by source, destination and port; allow, deny, block; automatic recovery; multiple clients
Dynamic SDN functionality	Rules installed and removed at run time; statistics polling

Rubric item	Where it is addressed
Challenging scenario	Flood, unauthorised access, controller failure
Performance evaluation	Static vs dynamic comparison at three intensities with an idle reference
Documentation	README, this overview, viva notes, tests

11. Limitations

- IPv4 and TCP/UDP only; one switch in the topology.
- Detection is per source IP, so a spoofed address would defeat it.
- A block removes all connectivity for a host; DENY rules give finer-grained responses.
- Thresholds are fixed per policy file; adaptive thresholds would reduce false positives.
- If the controller fails, installed rules stay in place but no new rules are installed until it returns.